Elementary graph algorithms

Part VI Graph Algorithms

Scope

Search order determines distances, recursion ancestry, and finish times. On one six-vertex directed graph, FIFO layers and the white-path theorem establish the correctness and linear cost of BFS, DFS, topological sorting, and SCC decomposition.

Prerequisites

Graph representations, colors, timestamps, and linear traversals.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Choose adjacency matrices or lists.
- Derive the shortest-path property of BFS.
- Use DFS for topological sorting and SCCs.
- 22.1 Representations of graphs
- 22.2 Breadth-first search
- 22.3 Depth-first search
- 22.4 Topological sort
- 22.5 Strongly connected components

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.1 Representations of graphs

Graphs here are finite and either directed or undirected, with stable vertex identifiers and explicitly specified edge directions. A path may have length zero, so every vertex reaches itself. Let $n=|V|$ and $m=|E|$, counting each undirected edge once. Unless stated otherwise, shortest distance counts **edges**, rather than arbitrary edge weights.

An adjacency list has n headers. Its lists contain m entries for a directed graph and 2m for an undirected simple graph. Space is $\Theta(n+m)$. Enumerating u's neighbors takes $\Theta(1+\text{outdegree}(u))$; testing (u,v) by scanning an ordinary unsorted list takes that same worst-case time. Hashing or sorting changes lookup guarantees and must not be assumed for an ordinary list.

In an adjacency matrix A, rows are sources and columns are destinations. $A[u,v]=1$ means $u \to v$. Space is $\Theta(n^2)$, edge lookup $\Theta(1)$, and enumerating u's neighbors generally scans n entries. An undirected graph satisfies $A=A^{\mathsf{T}}$. A weighted matrix needs a separate absence marker because zero can be a valid edge weight.

One six-vertex directed graph

BFS, DFS, and SCC below use **these same directed edges**, with outer vertex order and adjacency order as displayed.

Vertex	Adj[u]	outdegree	indegree
a	b,c	2	1
b	c	1	2
c	a	1	2
d	b,e	2	1
e	d	1	2
f	e	1	0

Here n=6,m=8. The list lengths sum to 8, as do the indegrees, rather than 16. To construct G^{T}, initialize n empty lists and scan all m entries, placing u in $\text{Adj}^{\mathsf{T}}[v]$ for every $u \to v$. Time and additional space are $\Theta(n+m)$; the original graph need not change.

One six-vertex directed graph (continued)

One graph as lists and a matrix

Adj[u]: row u → column v. Directed graph, m=8.

Vertex	Adjacency list
a	b,c
b	c
c	a
d	b,e
e	d
f	e

Columns of A

	a	b	c	d	e	f
a	0	1	1	0	0	0
b	0	0	1	0	0	0
c	1	0	0	0	0	0
d	0	1	0	0	1	0
e	0	0	0	1	0	0
f	0	0	0	0	1	0

Green cells are directed edges: 8 cells correspond to 8 list entries.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

22.2 Breadth-first search

BFS maintains distance layers with a FIFO queue. White means undiscovered, gray discovered but not fully scanned, and black fully scanned. The discovery predecessor is π[v]; ∞ means no known distance and NIL no predecessor. Neither can be replaced by zero.

```
BFS(G,s):
  for v in V: color[v]=WHITE; d[v]=∞; π[v]=NIL
  color[s]=GRAY; d[s]=0; Q=empty queue; enqueue(Q,s)
  while Q is not empty:
    u=dequeue(Q)
    for v in Adj[u]:
      if color[v]==WHITE:
        color[v]=GRAY; d[v]=d[u]+1; π[v]=u
        enqueue(Q,v)
    color[u]=BLACK
  return d,π
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

22.2 Breadth-first search (continued)

Change color before enqueueing, so competing predecessors cannot enqueue a vertex twice. At the start of each while iteration, the queue contains exactly the gray vertices. While scanning u, however, u is gray and already dequeued. The invariant does not hold after every individual instruction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why FIFO processes nearer vertices first

List the queue from front to back as $q_1,\ldots,q_r$. Distances are nondecreasing and $d[q_r] \leq d[q_1]+1$; regard this condition as true for an empty queue.

Initially only s is queued. If u is removed with $d[u]=k$, the remaining distances, by induction, are k or k+1 in nondecreasing order. Every new vertex receives distance k+1 and is appended, preserving order. If layer k empties, the next front has distance k+1; only a later iteration can generate distance k+2. Dequeued distances are therefore nondecreasing.

Every finite d[v] describes an actual predecessor path. The source has length zero, and discovering v appends u→v to u's path, giving length d[u]+1. Thus true shortest distance $\delta(s,v) \leq d[v]$. Distances decrease by one along predecessor links, so those links cannot cycle.

An actual path gives only an upper bound. Shortest-path correctness still requires ruling out an earlier discovery by a longer route.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete proof of BFS shortest distances

Suppose a reachable v ends with $d[v] \neq \delta(s,v)$, and choose such a v with minimum δ. It is not s. Let u precede v on a shortest path. Then $\delta(s,u)=\delta(s,v)-1$ and, by minimality, $d[u]=\delta(s,u)$. Vertex u has finite distance, is enqueued, and is eventually dequeued.

When $u \to v$ is scanned, if v is white, it receives $d[v]=d[u]+1=\delta(s,v)$, a contradiction. Otherwise let w have first discovered v. Vertex w was dequeued no later than u. Queue order gives $d[w] \leq d[u]$, hence $d[v]=d[w]+1 \leq \delta(s,v)$. Together with $\delta \leq d$ from the predecessor path, this again gives equality. Both cases contradict the choice of v. All reachable distances are correct. Discovering an unreachable vertex would give an actual path to it, also a contradiction, so its distance remains ∞.

Every reachable nonsource vertex has one predecessor, and following predecessors reaches s after exactly d[v] edges. These links form a BFS tree. Adjacency order can change the selected predecessors and shortest paths, but not distances.

Initialization takes $\Theta(n)$. Each vertex enters and leaves the queue at most once; each reachable adjacency list is scanned once. Total time is $O(n+m)$, with $O(n)$ additional space for the queue and arrays. A matrix scans n positions for every dequeued vertex, giving worst-case $\Theta(n^2)$.

Complete queue trace from d

The queue shown is its content **after scanning finishes** in that iteration. Parenthesized numbers are distances; newly discovered vertices receive predecessors immediately.

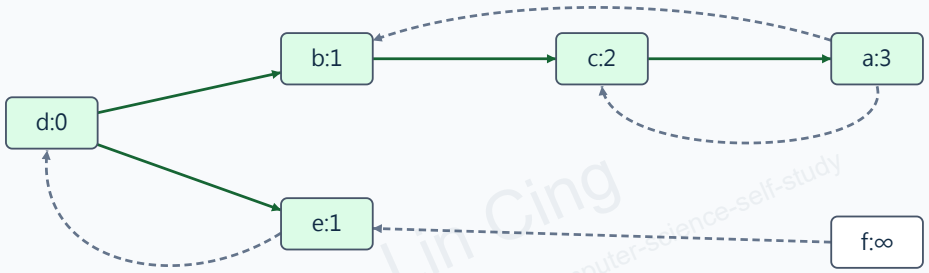
Removed u	Scan result	Queue at iteration end
Before the loop	Initialize d[d]=0	d(0)
d	b white: d[b]=1,π[b]=d; e white: d[e]=1,π[e]=d	b(1),e(1)
b	c white: d[c]=2,π[c]=b	e(1),c(2)
e	d already black; do not enqueue	c(2)
c	a white: d[a]=3,π[a]=c	a(3)
a	b,c already discovered; unchanged	empty

Finally $(d[a],d[b],d[c],d[d],d[e],d[f])=(3,1,2,0,1,\infty)$. A shortest path to a is $d \to b \to c \to a$. Edge $f \to e$ does not permit travel from d backward to f.

Complete queue trace from d (continued)

BFS from d on the same six-vertex graph

Boxes: vertex:distance. Green: predecessor edges. Gray dashed: other edges.



Dequeue: d(0) → b(1) → e(1) → c(2) → a(3). Vertex f stays at ∞.

Directed distances, equal weights, and path counts

For every edge $u \to v$ with reachable u, $\delta(s,v) \leq \delta(s,u)+1$. A directed graph provides no reverse inequality. Here $a \to b$ goes from distance 3 to 1, a difference of 2. Only when both directions of an undirected edge exist do we obtain $|d[u]-d[v]| \leq 1$.

For equal positive weight w, shortest weight is $w \cdot d[v]$. BFS fails for general weights: $s \to t$ of weight 9 competes with $s \to x$ and $x \to t$ of weight 1 each. The one-edge route costs 9, whereas the two-edge route costs 2. With all weights zero, every reachable route costs zero. Negative weights or cycles require a separately specified problem.

To count shortest paths in a unit-weight graph, initialize ways[s]=1 and all other counts to zero. Discover and enqueue white v as usual. For **every** edge $u \to v$ satisfying $d[v]=d[u]+1$, add ways[u] to ways[v], including edges into an already discovered v. Count each simple-graph edge once. All distance-$(k-1)$ vertices finish before distance-k vertices are dequeued, so ways[u] is complete when u is dequeued.

Directed distances, equal weights, and path counts (continued)

For edges 0→1,0→2,1→3,2→3,1→4,2→4,3→5,4→5, distances are (0,1,1,2,2,3), and ways= (1,1,1,2,2,4). This uses O(n+m) additions. Counts may occupy Θ(n) bits, so arbitrary-precision additions are not unconditionally constant-time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.3 Depth-first search

DFS completes the current recursive branch before advancing to another neighbor. The outer loop must cover all vertices, including those unreachable from the first root. Here time counts **discovery and finish events**, not scanned edges.

```
DFS(G,order):
  for v in V: color[v]=WHITE; π[v]=NIL
  time=0; finished=empty list
  for u in order:
    if color[u]==WHITE: Visit(u)
Visit(u):
  time=time+1; tin[u]=time; color[u]=GRAY
  for v in Adj[u]:
    if color[v]==WHITE: π[v]=u; Visit(v)
  color[u]=BLACK; time=time+1; tout[u]=time
  append(finished,u)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.3 Depth-first search (continued)

Each vertex is visited once and each adjacency entry scanned once, taking $\Theta(n+m)$ time. Recursion depth is at most n and additional space $O(n)$. An iterative implementation must retain the next neighbor or an explicit finish event in each stack frame. Pushing all neighbors at once with a visited flag does not automatically reproduce the same forest or finish times.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Discovery and finish intervals nest or separate

For distinct u,v, suppose tin[u]<tin[v]. If v is discovered before u finishes, Visit(u) remains on the stack. Vertex v is its descendant and must return first, giving tin[u]<tin[v]<tout[v]<tout[u]. If v is discovered after u finishes, tin[u]<tout[u]<tin[v]<tout[v], so the intervals are disjoint.

Thus tin[u]<tin[v]<tout[u]<tout[v] is impossible. Conversely, strict interval containment means u was still on the stack when v was discovered, so v is a proper descendant. This parenthesis theorem depends on last-in-first-out recursive returns.

DFS on the six-vertex graph in a,b,c,d,e,f order gives the table. Its parenthesis sequence is (a(b(c c)b)a)(d(e e)d)(f f); every timestamp from 1 through 12 appears exactly once.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Discovery and finish intervals nest or separate (continued)

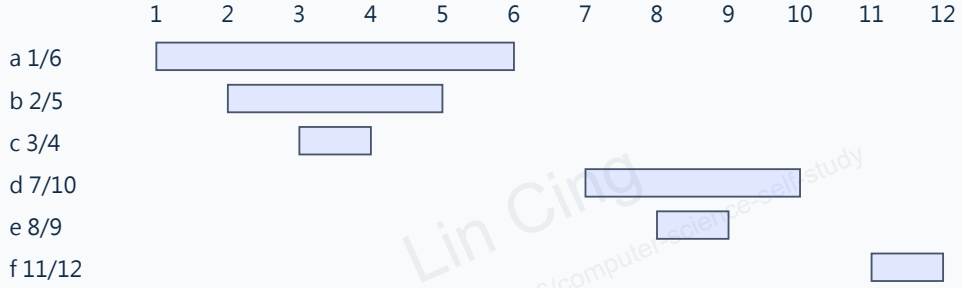
Vertex	π	tin	tout
a	NIL	1	6
b	a	2	5
c	b	3	4
d	NIL	7	10
e	d	8	9
f	NIL	11	12

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Discovery and finish intervals nest or separate (continued)

DFS discovery and finish events

Bars run from tin to tout. Descendants nest; different root intervals are disjoint.



a contains b,c; d contains e; f is the third root. Vertex intervals never interleave.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The time condition in the white-path theorem

Immediately **before Visit(u) changes u from white to gray**, v is a descendant of u in this DFS, allowing v=u, if and only if there is a u-to-v path whose vertices are all white at that instant. This time convention includes the starting vertex u itself.

For the forward implication, the forest path from u to v uses original graph edges. All proper descendants are discovered after tin[u], so every vertex on this path is white at the specified instant.

Conversely, take the first vertex y on a white path that supposedly fails to become a descendant of u. Its predecessor x is already such a descendant, possibly u. When Visit(x) scans x→y, a still-white y is recursively visited and becomes a descendant. If y has been discovered already, it was white at the initial instant and was therefore discovered during u's unfinished recursive call, also within u's subtree. Both cases contradict the assumption.

The requirement **all white at that instant** is essential. In a→b,a→c,c→b, scan b first. By c's discovery, b is black. Although c reaches b, b will not become a descendant of c. Ordinary reachability cannot replace the white-path condition.

Four edge types in directed DFS

While scanning u→v, u is the deepest gray vertex, and all gray vertices form exactly the stack's ancestor chain. White v gives a tree edge. Gray v gives a back edge, including a self-loop. For black v, tin[u]<tin[v] gives a forward edge; otherwise it is a cross edge.

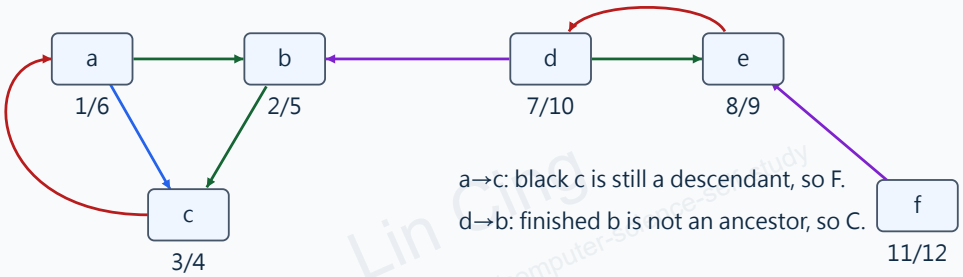
Type	Forest relationship	Timestamp check after DFS
tree	π[v]=u	tin[u]<tin[v]<tout[v]<tout[u]
forward	Nontree edge to a proper descendant	Same inequality; timestamps alone do not distinguish tree/forward
back	v is an ancestor of u, or v=u	For a proper ancestor: tin[v]<tin[u]<tout[u]<tout[v]
cross	Neither is the other's ancestor	tout[v]<tin[u]; preserve the scanned direction

The actual scan order is a→b(T),b→c(T),c→a(B),a→c(F),d→b(C),d→e(T),e→d(B),f→e(C). Edge a→c is scanned after c finishes. A black destination need not mean a cross edge.

Four edge types in directed DFS (continued)

Four directed edge types in one DFS

Green T: tree. Red B: back. Blue F: forward. Purple C: cross.



Labels are tin/tout. All 8 edges remain; their directions determine their types.

Undirected edge types and cycle tests

An undirected edge occurs in two adjacency lists. Classify the edge using the direction of its **first scan**. Suppose tin[u]<tin[v]. Since u lists v, v is discovered before u finishes. If u→v is the first scan, v must still be white, or v→u would already have been scanned; it is a tree edge. If v→u is scanned first, u is gray and it is a back edge. Thus undirected DFS has only tree/back edges, with no genuine forward/cross edges.

In a simple undirected graph, the reverse adjacency entry of the parent edge is not an additional cycle. With parallel edges, skip the same parent edge by **edge identifier**, rather than skipping all edges to the parent. A second parallel edge can form a length-two cycle under the multigraph convention.

A directed graph has a cycle if and only if DFS has a back edge. A back edge u→v, together with the tree path v to u, gives a cycle. Conversely, choose the earliest-discovered vertex v on a cycle. All other cycle vertices are white then. The cycle path to v's predecessor u is a white path, so the white-path theorem makes u a descendant of v. Edge u→v is a back edge. A self-loop satisfies the same conclusion directly.

22.4 Topological sort

A topological order permutes all vertices so every edge u→v satisfies pos[u]<pos[v]. A directed cycle would require strict increases around a loop back to the starting position, which is impossible. Exactly DAGs admit such an order.

Run DFS on a DAG and reverse its finish list; comparison sorting is unnecessary. For every scanned u→v, v cannot be gray, since that would give a cycle. If white, v becomes a descendant and tout[v] <tout[u]. If black, v has finished already and the same inequality holds. Decreasing finish order therefore directs every edge forward. Time is Θ(n+m), with O(n) additional space.

An implementation can report a cycle upon scanning a gray destination, and deliver the order only after validation. A reverse-finish list always exists, but it is not automatically a topological order for an arbitrary directed graph.

For a→c,b→c,c→d and outer order a,b,c,d, DFS gives a=1/6,c=2/5,d=3/4,b=7/8. Reverse finish order is b,a,c,d. Order a,b,c,d is also valid. **Order a,c,b,d is invalid because it reverses b→c.**

Kahn's indegree updates and failure condition

Compute indegrees and enqueue all zero-indegree vertices. Output a removed u, decrement indegree[v] for every u→v, and enqueue v exactly when its value becomes zero. The counts equal indegrees in the subgraph induced by vertices not yet output. Thus u has no unprocessed predecessor when output, and every output prefix respects edge order.

If a nonempty remainder has no zero-indegree vertex, repeatedly follow a remaining incoming edge from any vertex. Finiteness forces a repeated vertex and hence a cycle. Conversely, every nonempty induced subgraph of a DAG has a zero-indegree vertex, so all n vertices are output. Fewer than n outputs means failure. **A remaining vertex can lie downstream from a cycle without lying on one.**

Kahn's indegree updates and failure condition (continued)

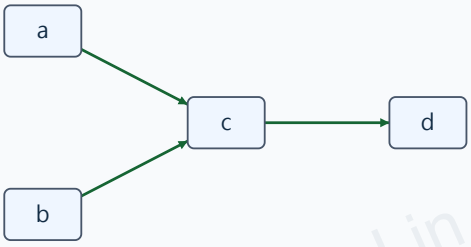
Output u	Remaining indegree of c	Remaining indegree of d	Queue at iteration end
Initial	2	1	a,b
a	1	1	b
b	0	1	c
c	0	0	d
d	0	0	empty

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kahn's indegree updates and failure condition (continued)

Kahn: indegrees and queue states

DAG: a→c, b→c, c→d. Remove a source, then decrement its destinations.



Operation	in(c)	in(d)	Queue
Initial	2	1	a,b
Remove a	1	1	b
Remove b	0	1	c
Remove c	0	0	d
Remove d	0	0	empty

Output a,b,c,d. Order a,c,b,d reverses b→c and is invalid.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.5 Strongly connected components

Define u~v exactly when u reaches v and v reaches u. Zero-length paths give reflexivity; the definition gives symmetry; concatenating paths gives transitivity. This equivalence relation partitions vertices into SCCs, the inclusion-maximal mutually reachable sets, **not merely the class with the greatest cardinality**.

The condensation has one vertex per SCC and an edge for every original edge between distinct classes. It is a DAG: a cycle of distinct classes would make all those classes mutually reachable, contradicting their separation. Reversing every edge reverses paths in both directions, so G and G^T have identical SCCs.

For the six-vertex graph, A={a,b,c},D={d,e},F={f}, with condensation F→D→A. In G^T the direction is A→D→F. Starting arbitrarily at a in G^T can reach all three SCCs. **Transposition alone does not prevent crossing components; root order and visited marks are essential.**

Why finish times order different SCCs

Use timestamps from the **first DFS on G**. Define tin(C)=min tin[v] and F(C)=max tout[v] over v∈C. For distinct SCCs with C→D, F(C)>F(D). There are two discovery cases.

1. If tin(C)<tin(D), the first discovered x in C can follow white paths inside C to the outgoing edge, cross to D, and traverse D internally. The white-path theorem puts every other C vertex and every D vertex below x. Thus F(C)=tout[x]>F(D). Paths between C vertices can be chosen entirely inside C: any outside vertex on such a path would also be mutually reachable with C and therefore belong to it.
2. If tin(D)<tin(C), the first y in D reaches every white vertex of D, so F(D)=tout[y]. No path leads from D to C, or C→D would make them one SCC. Hence y's recursion cannot discover C. C is discovered after y finishes, implying F(C)>F(D).

Why finish times order different SCCs (continued)

Consequently, a cross-component edge C→D in G^T satisfies F(C)<F(D). This compares **maximum finish times per component**, not every pair of individual finish times. For a↔b,a→c, scanning a→b first gives tout[b]=3,tout[c]=5,tout[a]=6. Then F({a,b})=6>F({c})=5 even though b finishes before c. The actual cross-component edge a→c still has tout[a]>tout[c]. An edge between different SCCs cannot be a back edge.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two complete DFS passes and the boundary proof

```
SCC(G):
  first=DFS(G,a fixed order of all vertices)
  roots=reverse(first.finished)    // save the first finish order
  GT=transpose(G)
  reset every vertex to WHITE, π=NIL for the second pass
  consider vertices in roots order; call Visit(u) on GT only if WHITE
  output each new root's entire DFS tree as one component
```

Inductively, every previously output tree is exactly one SCC. The next white root u lies in a component C with the greatest F(C) among all unprocessed components. If C's maximum-finish vertex were already processed, the induction hypothesis would make all of C processed, contradicting white u.

Paths internal to C reach every C vertex from u. All remaining C vertices are white, so the second pass collects all of them. For a cross edge C→D in G^T, F(D)>F(C), so D must already be black. Visit does not recurse into it, and cannot cross it to another white class. The tree collects neither too few nor too many vertices, completing the induction.

Two complete DFS passes and the boundary proof (continued)

Each DFS takes Θ(n+m), as does transposition. Reversing the appended finish list takes Θ(n); no O(n log n) sort is needed. Total time is Θ(n+m), and the transpose plus arrays use O(n+m) additional space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both passes on the same six-vertex graph

The first finish list is c,b,a,e,d,f, giving second-pass candidates f,d,e,a,b,c. The transposed lists, alphabetized internally, are a:[c],b:[a,d],c:[a,b],d:[e],e:[d,f],f:[].

Candidate root	New discoveries in G^T	Cross-component edge encountered	Output
f	f	none	{f}
d	d,e	e→f; f already black	{d,e}
e	skip, already black	none	no new component
a	a,c,b	b→d; d already black	{a,b,c}
b,c	skip, already black	none	no new component

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both passes on the same six-vertex graph (continued)

The values F(A)=6,F(D)=10,F(F)=12 decrease along the original condensation F→D→A. Reversing discovery order could place c ahead of a, but this particular example need not fail. Accidental success is not a correctness argument. The minimal counterexample is x→y: first-pass discoveries tin[x]=1,tin[y]=2 would start the second pass at y, whose transposed edge y→x incorrectly merges two SCCs.

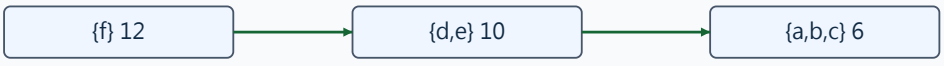
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Both passes on the same six-vertex graph (continued)

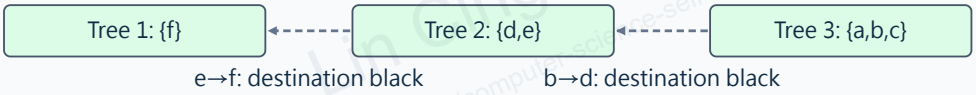
SCC: finish order prevents crossing in pass two

First-pass maxima in G: F({f})=12, F({d,e})=10, F({a,b,c})=6.

Original condensation DAG



Root order and black boundaries in GT



Green boxes collect one SCC each. Gray arrows cannot recurse into black vertices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 22.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study